

Trabajo Práctico Integrador

Programación I

Universidad tecnológica Nacional.

“Gestión de datos de Países en Python: Filtros, ordenamiento y estadísticas”

Alumnas:

Guzmán Natalia Andrea

Gelvez Florencia Jaqueline.

Docente: Ariel Enferrel, Martín García, Cinthia Rigoni.

tutor: Sarmiento Juan Ignacio

fecha de entrega:

14 de junio del 2026

índice	
introducción	3
Marco teórico	4
Decisiones técnicas y Arquitectura	10
conclusión	13
Links	13
Bibliografía/Webgrafía	14

1-Introducción

El presente Trabajo Práctico Integrador tiene como objetivo desarrollar una aplicación en Python para la gestión de información de países, utilizando estructuras de datos, funciones y archivos CSV. A través de este proyecto se busca aplicar los conocimientos adquiridos en la materia Programación 1, integrando conceptos fundamentales de programación y resolución de problemas.

La aplicación permite cargar información desde un archivo CSV, realizar búsquedas, aplicar filtros, ordenar registros y generar estadísticas sobre los datos almacenados. Para ello se emplean listas y diccionarios como estructuras principales de almacenamiento, junto con funciones que favorecen la modularidad y organización del código.

Además, se implementan mecanismos básicos de validación y manejo de errores con el propósito de garantizar un funcionamiento correcto del sistema ante distintas situaciones de uso. De esta manera, el proyecto combina aspectos teóricos y prácticos, permitiendo afianzar habilidades relacionadas con el desarrollo de software y el tratamiento de datos.

2- Marco Teórico

Listas:

Las listas son secuencias mutables, que se utilizan normalmente para almacenar colecciones de elementos homogéneos (donde el grado exacto de similitud variará según la aplicación).

Las listas se pueden construir de varias maneras:

- Utilizando un par de corchetes para indicar la lista vacía: []
- Utilizando corchetes y separando los elementos con comas: [a],[a, b, c]
- Utilizando una comprensión de lista:[x for x in iterable]
- Usando el constructor de tipo: list()o list(iterable)

El constructor crea una lista cuyos elementos son los mismos y están en el mismo orden que los elementos del iterable . El iterable puede ser una secuencia, un contenedor que admite iteración o un objeto iterador. Si el iterable ya es una lista, se crea una copia y se devuelve, de forma similar a iterable[:]. Por ejemplo, list('abc') devuelve y devuelve . Si no se proporciona ningún argumento, el constructor crea una nueva lista vacía, .['a', 'b', 'c']list((1, 2, 3))[1, 2, 3][].

```
def cargar_paises():  
    paises = []
```

Ejemplo extraído del programa realizado en el presente trabajo.

Diccionario:

Estos representan conjuntos finitos de objetos indexados por valores casi arbitrarios. Son mutables, esto quiere decir que pueden ser modificados; pueden ser creados por la notación {}

Los diccionarios tienen claves que conservan el orden de inserción, lo que significa que las claves se mantendrán en el mismo orden en que se agregaron secuencialmente sobre el diccionario. Reemplazar una clave existente no cambia el orden, sin embargo, eliminar una clave y volver a insertarla la agregará al final en lugar de mantener su lugar anterior.

```
países.append({  
    "nombre": nombre,  
    "poblacion": poblacion,  
    "superficie": superficie,  
    "continente": continente  
})
```

Ej:

Diccionario con 4 claves. Corresponde al programa generado para la gestión de datos de países.

Funciones:

Las funciones sirven para agrupar instrucciones que realizan una tarea específica y que pueden reutilizarse varias veces sin tener que escribir el mismo código de nuevo.

El intérprete de Python tiene una serie de funciones y tipos incluidos en él que están siempre disponibles. Están listados aquí en orden alfabético.

Funciones incorporadas

A

[abs\(\)](#)
[aiter\(\)](#)
[all\(\)](#)
[anext\(\)](#)
[any\(\)](#)
[ascii\(\)](#)

B

[bin\(\)](#)
[bool\(\)](#)
[breakpoint\(\)](#)
[bytearray\(\)](#)
[bytes\(\)](#)

C

[callable\(\)](#)
[chr\(\)](#)
[classmethod\(\)](#)
[compile\(\)](#)
[complex\(\)](#)

D

[delattr\(\)](#)
[dict\(\)](#)
[dir\(\)](#)
[divmod\(\)](#)

E

[enumerate\(\)](#)
[eval\(\)](#)
[exec\(\)](#)

F

[filter\(\)](#)
[float\(\)](#)
[format\(\)](#)
[frozenset\(\)](#)

G

[getattr\(\)](#)
[globals\(\)](#)

H

[hasattr\(\)](#)
[hash\(\)](#)
[help\(\)](#)
[hex\(\)](#)

I

[id\(\)](#)
[input\(\)](#)
[int\(\)](#)
[isinstance\(\)](#)
[issubclass\(\)](#)
[iter\(\)](#)

L

[len\(\)](#)
[list\(\)](#)
[locals\(\)](#)

M

[map\(\)](#)
[max\(\)](#)
[memoryview\(\)](#)
[min\(\)](#)

N

[next\(\)](#)

O

[object\(\)](#)
[oct\(\)](#)
[open\(\)](#)
[ord\(\)](#)

P

[pow\(\)](#)
[print\(\)](#)
[property\(\)](#)

R

[range\(\)](#)
[repr\(\)](#)
[reversed\(\)](#)
[round\(\)](#)

S

[set\(\)](#)
[setattr\(\)](#)
[slice\(\)](#)
[sorted\(\)](#)
[staticmethod\(\)](#)
[str\(\)](#)
[sum\(\)](#)
[super\(\)](#)

T

[tuple\(\)](#)
[type\(\)](#)

V

[vars\(\)](#)

Z

[zip\(\)](#)

[__import__\(\)](#)

```
def cargar_paises():
    paises = []
```

EJ:

```
def mostrar_paises(lista):
    if len(lista) == 0:
        print("No hay datos para mostrar.")
        return
```

Ejemplos correspondientes al programa generado .

Condicionales:

Las estructuras condicionales o estructuras de decisión permiten que un programa ejecute diferentes bloques de instrucciones según se cumpla o no una determinada condición lógica. En Python estas estructuras se implementan mediante las sentencias `if`, `elif` y `else`.

Son expresiones que devuelven el valor A o el valor B dependiendo de si un valor booleano es verdadero o falso. Una expresión condicional le permite escribir una única sentencia de asignación que tiene el mismo efecto que la siguiente:

```
if len(resultado) > 0:
    print(f"\nPaíses con población entre {minimo} y {maximo}:")
    mostrar_paises(resultado)
else:
    print("No se encontraron países en ese rango de población.")
```

Ejemplos extraídos del programa realizado en el presente trabajo.

Ordenamientos:

Una clasificación ascendente simple es muy fácil: simplemente llame a la función [`sorted\(\)`](#). Retorna una nueva lista ordenada:

```
>>> sorted([5, 2, 3, 1, 4])
[1, 2, 3, 4, 5]
```

También puede usar el método [`list.sort\(\)`](#). Modifica la lista in situ (y retorna `None` para evitar confusiones). Por lo general, es menos conveniente que [`sorted\(\)`](#), pero si no necesita la lista original, es un poco más eficiente.

```
>>> a = [5, 2, 3, 1, 4]
>>> a.sort()
>>> a
[1, 2, 3, 4, 5]
```

Otra diferencia es que el método [`list.sort\(\)`](#) solo aplica para las listas. En contraste, la función [`sorted\(\)`](#) acepta cualquier iterable.

```
>>> sorted({1: 'D', 2: 'B', 3: 'B', 4: 'E', 5: 'A'})
[1, 2, 3, 4, 5]
```

Ej:

```
# Ordenamos usando la función obtener_nombre
ordenados = sorted(paises, key=obtener_nombre, reverse=reversa)
mostrar_paises(ordenados)
```

Ejemplos extraídos del programa realizado en el presente trabajo.

Estadísticas Básicas:

Estas funciones calculan el promedio o el valor típico de distintos elementos y manejan objetos como `int`, `float`, `Decimal` y `Fraction`.

Su comportamiento con colecciones mixtas que contengan objetos de diferente tipo no está definido y depende de la implementación.

Cuando los datos de entrada consisten en una mezcla de varios tipos, puedes usar `map()` para asegurarte de que el resultado sea consistente, por ejemplo: `map(float, input_data)`.

Estas funciones calculan el promedio o el valor típico de una población o muestra.

<code>mean()</code>	Media aritmética («promedio») de los datos.
<code>fmean()</code>	Media aritmética usando coma flotante, más rápida.
<code>geometric_mean()</code>	Media geométrica de los datos.
<code>harmonic_mean()</code>	Media armónica de los datos.
<code>median()</code>	Mediana (valor central) de los datos.
<code>median_low()</code>	Mediana baja de los datos.
<code>median_high()</code>	Mediana alta de los datos.
<code>median_grouped()</code>	Mediana, o percentil 50, de los datos agrupados.
<code>mode()</code>	Moda única (valor más común) de datos discretos o nominales.
<code>multimode()</code>	Lista de modas (valores más comunes) de datos discretos o nominales.
<code>quantiles()</code>	Divide los datos en intervalos equiprobables.

Ejemplos proporcionados por la página oficial de Python.

```
opcion = input("Seleccione una opción: ").strip()
```

Ejemplos extraídos del programa realizado en el presente trabajo.

Archivo CVS:

Un archivo es una unidad de almacenamiento que contiene datos que pueden ser leídos o escritos por un programa. Python permite abrir archivos, leer su contenido, modificarlos y guardarlos, de forma muy flexible.

El tan llamado CSV (Valores Separados por Comas) es el formato más común de importación y exportación de hojas de cálculo y bases de datos. El formato CSV se utilizó durante muchos años antes de intentar describir el formato de manera estandarizada en [RFC 4180](#). La falta de un estándar bien definido significa que a

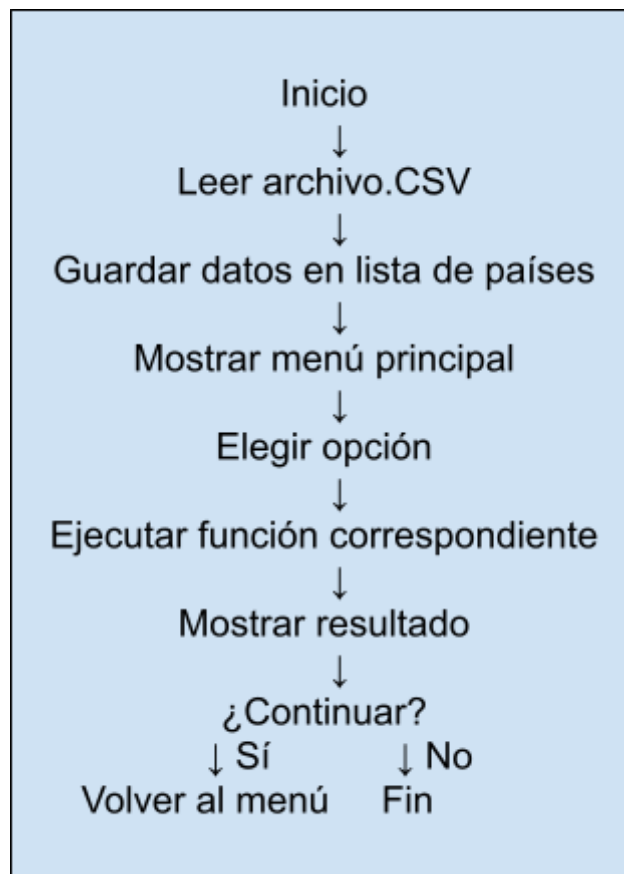
veces existen pequeñas diferencias en la información producida y consumida por diferentes aplicaciones. Estas diferencias pueden ser molestas al momento de procesar archivos CSV desde múltiples fuentes. Aún así, aunque los delimitadores y separadores varíen, el formato general es lo suficientemente similar como para que sea posible un sólo módulo que puede manipular tal información eficientemente, escondiendo los detalles de lectura y escritura de datos del programador.

```
ARCHIVO_CSV = "países.csv"
```

Ejemplos extraídos del programa realizado en el presente trabajo.
Se importa el archivo CSV con la información necesaria para que el programa funcione correctamente.

Flujo de operaciones en un diagrama o esquema:

Representa la secuencia ordenada de acciones, tareas o procesos que se ejecutan para cumplir el objetivo principal de un sistema. En un diagrama o esquema, muestra el recorrido normal de la información y las decisiones desde el inicio hasta la finalización del proceso.



3. Decisiones Técnicas y Arquitectura

Estructura de Datos Utilizada

Para almacenar la información de los países se eligió una estructura basada en una lista de diccionarios. Cada diccionario representa un país y almacena sus principales atributos: nombre, población, superficie y continente.

Ejemplo de un registro utilizado por el sistema:

```
{  
  "nombre": "Ecuador",  
  "poblacion": 18300000,  
  "superficie": 256370,  
  "continente": "America"  
}
```

Todos los registros son almacenados dentro de una lista principal denominada **países**, lo que permite recorrer fácilmente la información para realizar búsquedas, filtros, ordenamientos y cálculos estadísticos.

Almacenamiento y Persistencia de Datos

Para garantizar la conservación de la información entre distintas ejecuciones del programa, se decidió utilizar un archivo CSV denominado **países.csv**. El nombre del archivo se almacena en una constante global:

```
ARCHIVO_CSV = "países.csv"
```

Al iniciar el sistema, los datos son cargados desde el archivo CSV mediante la función **cargar_paises()**. Asimismo, cada vez que se agrega o actualiza un país, la información es guardada nuevamente utilizando la función **guardar_paises()**. Esta estrategia asegura que los cambios realizados por el usuario permanezcan almacenados para futuras ejecuciones.

Validación de Datos

Uno de los aspectos considerados durante el desarrollo fue la validación de la información ingresada por el usuario y de los datos provenientes del archivo CSV.

Para ello se implementaron funciones específicas:

```
def leer_cadena_no_vacia()
```

```
def leer_entero_positivo()
```

Estas funciones permiten controlar que los campos obligatorios no queden vacíos y que los valores numéricos ingresados sean válidos y mayores o iguales a cero. Además, durante la lectura del archivo CSV se verifican posibles errores de formato, evitando cargar registros incompletos o incorrectos.

Modularización del Sistema

Con el objetivo de mejorar la organización y facilitar el mantenimiento del código, el programa fue dividido en funciones independientes, cada una encargada de una tarea específica.

Entre las principales funciones desarrolladas se encuentran:

- `cargar_paises()`
- `guardar_paises()`
- `agregar_pais()`
- `actualizar_pais()`
- `buscar_pais()`
- `filtrar_continente()`
- `filtrar_poblacion()`
- `filtrar_superficie()`
- `ordenar_nombre()`
- `ordenar_poblacion()`
- `ordenar_superficie()`
- `mostrar_estadisticas()`

Esta división permite reutilizar código, simplificar futuras modificaciones y mejorar la legibilidad del programa.

Procesamiento y Ordenamiento de Datos

Para las operaciones de ordenamiento se utilizó la función integrada `sorted()`. Con el fin de mantener una estructura clara y reutilizable, se crearon funciones auxiliares que indican el criterio de ordenamiento:

```
def obtener_nombre(pais):  
    return pais["nombre"].lower()  
  
def obtener_poblacion(pais):  
    return pais["poblacion"]  
  
def obtener_superficie(pais):  
    return pais["superficie"]
```

De esta manera, el sistema permite ordenar los registros por nombre, población o superficie, tanto en forma ascendente como descendente.

Arquitectura General del Sistema

El funcionamiento general del programa sigue una arquitectura basada en menús. Al iniciar la aplicación, se cargan los datos desde el archivo CSV y se presenta un menú principal desde el cual el usuario puede acceder a todas las funcionalidades disponibles.

Las principales operaciones son:

- Agregar países.
- Actualizar información existente.
- Buscar países.
- Aplicar filtros por continente, población o superficie.
- Ordenar registros.
- Obtener estadísticas generales.
- Visualizar la lista completa de países.

Una vez finalizada cada operación, el sistema retorna al menú principal hasta que el usuario selecciona la opción de salida. Esta arquitectura facilita la interacción con el usuario y permite una navegación sencilla dentro del programa.

```
print("\n===== GESTIÓN DE PAÍSES =====")
print("1. Agregar país")
print("2. Actualizar país")
print("3. Buscar país")
print("4. Filtrar países")
print("5. Ordenar países")
print("6. Estadísticas")
print("7. Mostrar todos")
print("0. Salir")
```

Dificultades y conclusión:

Durante el desarrollo de la investigación se presentaron diversas dificultades relacionadas con la búsqueda y selección de información confiable. Uno de los principales desafíos fue encontrar definiciones técnicas respaldadas por fuentes oficiales y académicas, ya que gran parte de la información disponible en Internet proviene de blogs, foros o sitios no especializados.

Además, algunos conceptos, como el flujo de operaciones principales y las estructuras de control, no siempre aparecen definidos de manera explícita en la documentación oficial de Python, por lo que fue necesario analizar diferentes fuentes y relacionar la información encontrada con los contenidos estudiados en la materia. También se requirió interpretar documentación técnica escrita para programadores y adaptarla a un lenguaje más claro y comprensible para el trabajo práctico.

Por último, fue necesario verificar que las fuentes consultadas fueran confiables y reconocidas, utilizando documentación oficial de Python y material académico proveniente de instituciones educativas de prestigio como la Universidad Tecnológica Nacional (UTN).

Links:

Repositorio GitHub: https://github.com/Natti92/TP_Python_Paises

Video explicativo:

https://drive.google.com/file/d/1N8cS9fdKW_u1r7pYuuqjQAYDaFjYfyow/view?usp=sharing

Fuentes oficiales utilizadas para la investigación:

Pagina oficial de Python:

<https://docs.python.org/es/3/search.html?q=Ordenamientos>

Repositorio institucional de la Universidad Tecnológica Nacional:

<https://ria.utn.edu.ar/home>

material Algoritmos y Estructuras de Datos; Programación I